VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

WEB APPLICATION DEVELOPMENT
IT093IU
LAB 3

Member List:

- Huỳnh Ngọc Bảo Long – ITITWE23023

Instructor: MSc. Nguyễn Trung Nghĩa

I. Task 1.1

1. Validation functions

a. 'validateUsername(username)'

- Check if the username input is valid or not.
- Functionality:
  - Defines a regex that allows only letters (upper or lower case) and digits, as well as requires the length to be between 4 – 20 characters. '^' and '$' anchor the pattern to the full string.
  - Uses '.trim()' removes leading or trailing whitespace and rejects empty or all space (blank) input.
  - Uses the regex '.test()' method to return 'true' if the username matches the pattern, otherwise 'false'.

```
function validateUsername(username) {
    // TODO: Check length (4-20) and alphanumeric
    const usernameRegex = /^[a-zA-Z0-9]{4,20}$/;
    if (username.trim() === "") return false;
    return usernameRegex.test(username);
}
```

- The empty check gives a more clear "blank" rejection immediately, the regex enforces format and length in a single and compact rule.

b. 'validateEmail(email)'

- Verify that the email fits the conventional format of a normal email
- Functionality:
  - Rejects blank input.
  - Regex meaning:
    - '[^\s@]+' → one or more characters that are not whitespace and not the '@' symbol.
    - @ → literal '@' sign, this ensures that following the first string is a '@' symbol as required by normal email formats.
    - '[^\s@]+' → domain part of the email (no spaces or @).
    - '\.' → literal dot.
    - '[^\s@]+' → top-level domain (com, net, etc.).
    - Anchors ^ and $ ensure the entire string matches this shape.
  - Returns Boolean whether the pattern matches.

```
function validateEmail(email) {
    // TODO: Use regex for email validation
    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    if (email.trim() === "") return false;
    return emailRegex.test(email);
}
```

- The regex used for this function covers common, practical invalid/valid cases but does not implement the full RFC edge cases (rarely needed for validation form)

c. 'validatePassword(password)'

- Ensure the password meets complexity and length requirements.
- Functionality:
  - Rejects blank password (and inputs that are only spaces)
  - Regex breakdown:
    - '(?=.*[A-Z])' → positive lookahead: there must be at least one uppercase letter anywhere.
    - '(?=.*\d)' → positive lookahead: there must be at least one digit.
    - '.{8,}' → at least 8 characters total (any character).
    - Anchors ^ and $ ensure full-string match.
  - Returns 'true' if password meets these constraints, otherwise 'false'.

```
function validatePassword(password) {
    // TODO: Check length >= 8, has uppercase, has number
    const passwordRegex = /^(?=.*[A-Z])(?=.*\d).{8,}$/;
    if (password.trim() === "") return false;
    return passwordRegex.test(password);
}
```

- Why lookaheads: They let you require presence of character classes (uppercase, number) without forcing order.

d. 'validatePasswordMatch(pass1, pass2)'

- Confirm the two password fields match and are not blank
- Functionality:
  - If either is blank (or only spaces), return 'false'.

- Returns ‘true’ only when the strings are exactly equal (including case and all characters).

```
function validatePasswordMatch(pass1, pass2) {
    // TODO: Compare passwords
    if (pass1.trim() === "" || pass2.trim() === "") return false;
    return pass1 === pass2;
}
```

- Prevents blank matching (two empty fields would equal each other) and ensures exact match.

e. ‘showError(fieldId, message)’

- Visually mark a field invalid and show its error message.
- Functionality:
  - Selects the ‘ ’ that holds the error text (e.g. ‘usernameError’)
  - Selects the input element itself (e.g. ‘username’)
  - Puts the provided message into the error container.
  - Adds ‘.show’ so CSS switches ‘.error-message’ from ‘display: none’ to visible
  - Removes ‘valid’ class (if any) and adds ‘invalid’, which triggers the red border via CSS

```
function showError(fieldId, message) {
    // TODO: Display error message
    const errorElement = document.getElementById(fieldId + "Error");
    const inputElement = document.getElementById(fieldId);
    errorElement.textContent = message;
    errorElement.classList.add("show");
    inputElement.classList.remove("valid");
    inputElement.classList.add("invalid");
}
```

- Centralize the UI changes for invalid state so all fields can use the same behavior.

f. ‘clearError(fieldId)’

- Mark a field valid and hide any error messages.
- Functionality:
  - Gets the error message node.
  - Gets the input node.
  - Clears error text.
  - Hides the error element.

- Gives the input a ‘valid’ class (green border) and ensures ‘invalid’ is removed

```
function clearError(fieldId) {
    // TODO: Clear error message
    const errorElement = document.getElementById(fieldId + "Error");
    const inputElement = document.getElementById(fieldId);
    errorElement.textContent = "";
    errorElement.classList.remove("show");
    inputElement.classList.remove("invalid");
    inputElement.classList.add("valid");
}
```

- Keeps valid/invalid visual feedback consistent and centralized.

g. ‘validateForm()’

- Validate all the fields at once, update error displays and enable/disable the submit button (enable when all input are valid and vice versa).
- Functionality:
  - Reads current input values.
  - Initialize ‘let valid = true’ – assumes the form is valid until a check fails.
  - Calls each validator in turn and:
    - If validation fails, calls ‘showError()’ with either a blank-specific message or a format message and sets ‘valid = false’.
    - If validation passes, calls ‘clearError()’ for that field.
  - After all fields are checked, sets the submit button

```
function validateForm() {
    // TODO: Validate all fields and enable/disable submit
    const username = document.getElementById("username").value;
    const email = document.getElementById("email").value;
    const password = document.getElementById("password").value;
    const confirmPassword = document.getElementById("confirmPassword").value;

    let valid = true;

    if (!validateUsername(username)) {
        showError("username", username.trim() === "" ? "Username cannot be blank." : "Username must be 4-20 alphanumeric characters
        valid = false;
    } else clearError("username");

    if (!validateEmail(email)) {
        showError("email", email.trim() === "" ? "Email cannot be blank." : "Enter a valid email address.");
        valid = false;
    } else clearError("email");

    if (!validatePassword(password)) {
        showError("password", password.trim() === "" ? "Password cannot be blank." : "Password must be at least 8 characters with 1
        valid = false;
    } else clearError("password");
```

```
    if (!validatePasswordMatch(password, confirmPassword)) {
        showError("confirmPassword", confirmPassword.trim() === "" ? "Please confirm your password." : "Passwords do not match.");
        valid = false;
    } else clearError("confirmPassword");

    document.getElementById("submitBtn").disabled = !valid;
}
```

- Why: A single function updates the UI consistently and decides whether submission is allowed

2. **Event listeners for real-time validation**
   - **Purpose and behavior:**
     - Each ‘input’ event fires whenever the user types, deletes, or pastes into the field.
     - ‘validateForm’ runs immediately and updates the UI — this gives instantaneous feedback and toggles the submit button dynamically.

```
// TODO: Add event listeners for real-time validation
document.getElementById("username").addEventListener("input", validateForm);
document.getElementById("email").addEventListener("input", validateForm);
document.getElementById("password").addEventListener("input", validateForm);
document.getElementById("confirmPassword").addEventListener("input", validateForm);

document.getElementById('signupForm').addEventListener('submit', function(e) {
    e.preventDefault();
    // TODO: Handle form submission
    alert("Form submitted successfully.");
});
```

- Why implemented: uses ‘input’ instead of ‘change’ or ‘blur’ because ‘input’ responds immediately, ‘change’ fires only when the field loses focus and its value changed, and ‘blur’ is only when the field loses focus. ‘input’ delivers the smoothest real-time UX.

II. Task 1.2

1. **‘addToCart(productId)’**
   - Adds a product to the cart or increases its quantity if it’s already in the cart.

```
function addToCart(productId) {
    // TODO: Add product to cart or increase quantity
    const product = products.find(p => p.id === productId);
    const existingItem = cart.find(item => item.id === productId);
    if (existingItem) {
        existingItem.quantity += 1;
    } else {
        cart.push({ ...product, quantity: 1 });
    }
    renderCart();
}
```

- Functionality:
  - 'find()' locates the selected product from the 'products' array
  - If it exists → increases its 'quantity' by 1
  - If not → push a new object (copy of 'product' + quantity = 1)
  - Calls 'renderCart()' to refresh the cart display and total

2. 'removeFromCart(itemId)'

- Removes a specific product from the cart entirely

```
function removeFromCart(itemId) {
    // TODO: Remove item from cart
    cart = cart.filter(item => item.id !== itemId);
    renderCart();
}
```

- Functionality:
  - Uses 'filter()' to return a new array excluding the product with that 'id'
  - Updates the 'cart' variable with this new filtered list
  - Calls 'renderCart()' to re-display the updated cart

3. 'updateQuantity(productId, change)'

- Increase or decreases the quantity of an item in the cart

```
function updateQuantity(productId, change) {
    // TODO: Update item quantity (change is +1 or -1)
    const item = cart.find(i => i.id === productId);
    if (!item) return;
    item.quantity += change;
    if (item.quantity <= 0) {
        removeFromCart(productId);
    }
    renderCart();
}
```

- Functionality:
  - Find the item in the ‘cart’ by ID
  - Adds the ‘change’ value (either +1 or -1)
  - If the quantity drops to 0 or below, the item is removed entirely
  - Calls ‘renderCart()’ to update the display and total

4. ‘calculateTotal()’

- Computes the total cost of all items in the cart

```
function calculateTotal() {
    // TODO: Calculate total price
    return cart.reduce((sum, item) => sum + item.price * item.quantity, 0);
}
```

- Functionality:
  - Uses ‘reduce()’ to iterate over all items in the ‘cart’
  - Adds up all subtotals into a single total price
  - Returns the final sum (number value)

5. ‘renderProducts()’

- Displays all product in the main “Products” grid

```
function renderProducts() {
    // TODO: Display products
    const grid = document.getElementById('productsGrid');
    grid.innerHTML = '';
    products.forEach(product => {
        const card = document.createElement('div');
        card.className = 'product-card';
        card.innerHTML =
        `
            <div class="product-image">${product.image}</div>
            <div class="product-name">${product.name}</div>
            <div class="product-price">$${product.price.toFixed(2)}</div>
            <button class="add-to-cart-btn" onclick="addToCart(${product.id})">Add to Cart</button>
        `;
        grid.appendChild(card);
    });
}
```

- Functional:
  - Selects the container '#productsGrid'.
  - Clears any existing content '(innerHTML = '')'.
  - Iterates through each 'product' in the 'products' array.
  - Creates a new 'div' element ('product-card') for each product.
  - Sets its inner HTML (emoji image, name, price, and "Add to Cart" button).
  - Appends the card to the product grid.
  - When the user clicks the button, 'addToCart(product.id)' runs.

6. 'renderCart()'

- Renders all cart items, updates totals and refreshes the cart count badge

```
function renderCart() {
    // TODO: Display cart items
    const cartItemsDiv = document.getElementById('cartItems');
    const cartCount = document.getElementById('cartCount');
    const cartTotal = document.getElementById('cartTotal');

    cartItemsDiv.innerHTML = '';
    cart.forEach(item => {
        const div = document.createElement('div');
        div.className = 'cart-item';
        div.innerHTML = `
            <div>${item.image} ${item.name}</div>
            <div>$${item.price.toFixed(2)}</div>
            <div class="quantity-controls">
                <button onclick="updateQuantity(${item.id}, -1)">-</button>
                <span>${item.quantity}</span>
                <button onclick="updateQuantity(${item.id}, 1)">+</button>
            </div>
            <div>$${(item.price * item.quantity).toFixed(2)}</div>
            <button onclick="removeFromCart(${item.id})">Remove</button>
        `;
        cartItemsDiv.appendChild(div);
    });
```

- Functionality:
  - Clears current cart display
  - Loops through each ‘item’ in ‘cart’
  - For each:
    - Shows product emoji and name
    - Shows individual price
    - Adds buttons for increasing/decreasing quantity
    - Displays subtotal (price x quantity)
    - Adds a ‘Remove’ button to remove the product
  - Updates the red badge (‘#cartCount’) to show total quantity
  - Updates total price in ‘#cartTotal’

7. ‘toggleCart()’

- Shows or hides the shopping cart section when the user clicks the cart icon

```
function toggleCart() {
    // TODO: Show/hide cart section
    const section = document.getElementById('cartSection');
    section.style.display = section.style.display === 'none' ? 'block' : 'none';
}
```

- Functionality:
  - Checks the current ‘display’ style of the cart section

- If it’s ‘none’ it sets it to ‘block’ to show it
- If it’s visible, it hides it again